



Lecture 22: Passing Data with Props

INTERVIEW QUESTION

> CORE

How does data flow between components in Svelte?

Your app has a list of user profiles, but the profile display logic is locked inside a reusable `<UserProfile />` component. How do you hand the component the specific user data it needs to render?

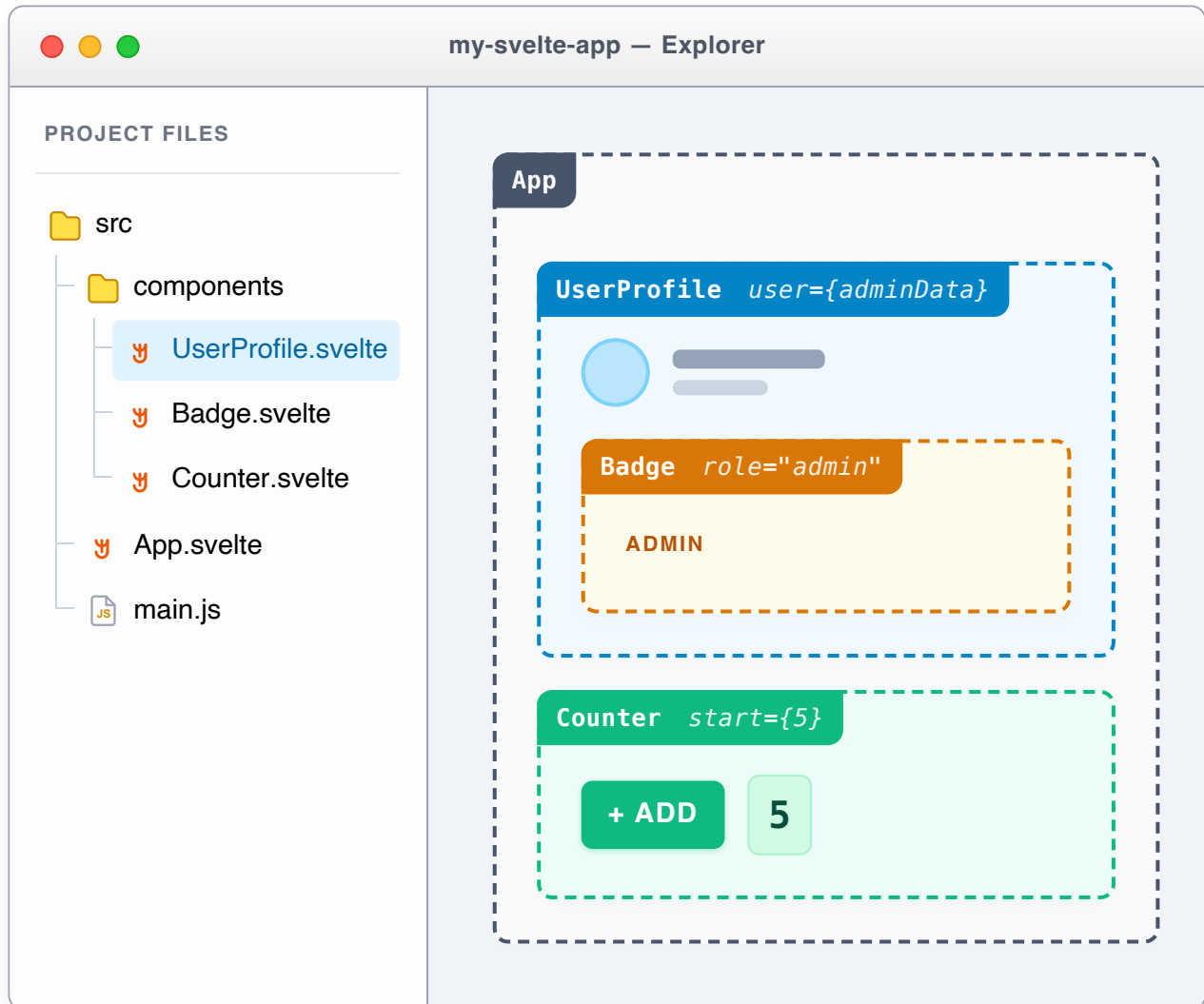
The Component Boundary

You've broken your interface down into pieces. A large `App.svelte` has delegated the job of rendering an individual user to a smaller file named `UserProfile.svelte`. This is the cornerstone of component-driven architecture: isolating responsibility. But isolation comes with a boundary. By default, `UserProfile.svelte` knows absolutely nothing about the data living inside `App.svelte`.

To cross this boundary, we need a controlled bridge. We need a way for the parent (the caller) to hand information down to the child (the callee). In UI frameworks, these inputs are called **props** (short for properties).

The File Structure

Before we write the code, let's look at the physical architecture. Components are just files on your computer. A parent component imports a child component from a folder, creating a relationship.



Data Flow Rules

Prop Mutability

Should a child component alter the data handed to it by the parent?

DON'T MUTATE

Avoid redefining or forcefully changing props inside the child. It breaks the one-way data flow model and makes state hard to track.

```
let { count } = $props(); count = 99; // ❌
```

Overwriting prop



DERIVE OR CALLBACK

If you need formatted data, use a `$derived` state. If you need to tell the parent to change it, pass a function prop or use events.

```
let { count } = $props(); let doubled =  
$derived(count * 2); // ✅
```